

UNIT 3

Pipelining

Computer Organization and Architecture (CSE23CT201)

9 hours | B.Tech CSE - Semester III

Learning Objectives

- ▶ Explain how pipelining overlaps the execution of multiple instructions to improve throughput.
- ▶ Describe the role cache memory plays in keeping a pipeline fed without stalling.
- ▶ Identify data hazards and instruction (control) hazards, and the techniques used to resolve them.
- ▶ Evaluate pipeline performance using speedup and efficiency measures.
- ▶ Discuss how instruction set design and datapath/control choices influence pipelining.

Maps to CO3: Learning the concept of pipelining for consistent execution of instructions with minimum hazards.



PART A

The Pipelining Idea

From Sequential Execution to Overlap

- Recall Unit 2: a single instruction executes in 5 steps - Fetch (F), Decode (D), Compute operand address, Fetch operand, Execute and write-back.
- If each step uses different hardware, the next instruction's Fetch can start while the current instruction is still in Decode - just like an assembly line.
- This overlapped execution of instructions across dedicated hardware stages is called pipelining.
- Goal: increase instruction throughput (instructions completed per unit time), not the time taken by any one instruction.

Ideal Overlap of Successive Instructions

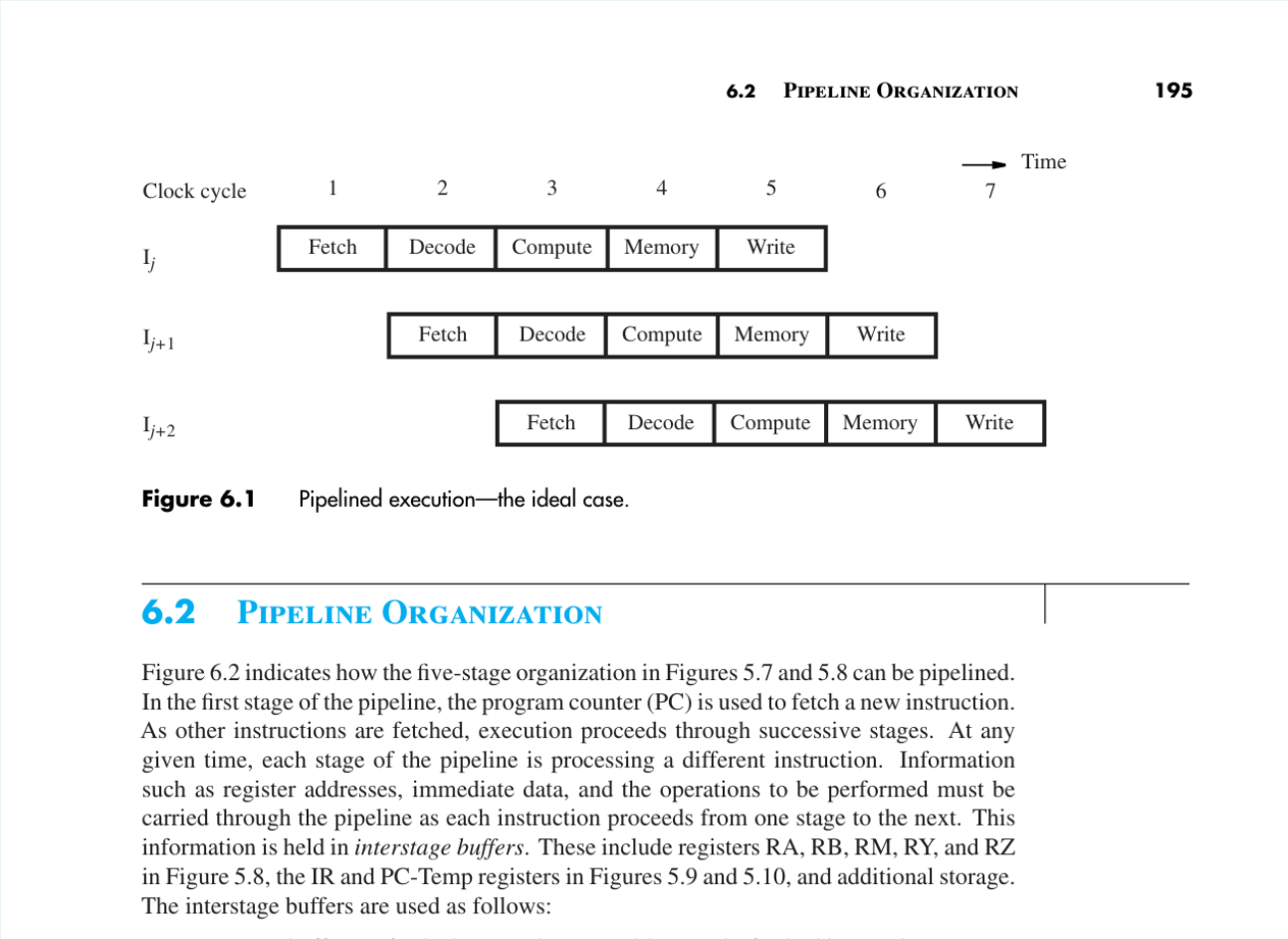


Figure 6.1 - Three instructions overlapped across stages: while one is executing, the next is decoding, and the next after that is fetching.

A Five-Stage Pipelined Datapath

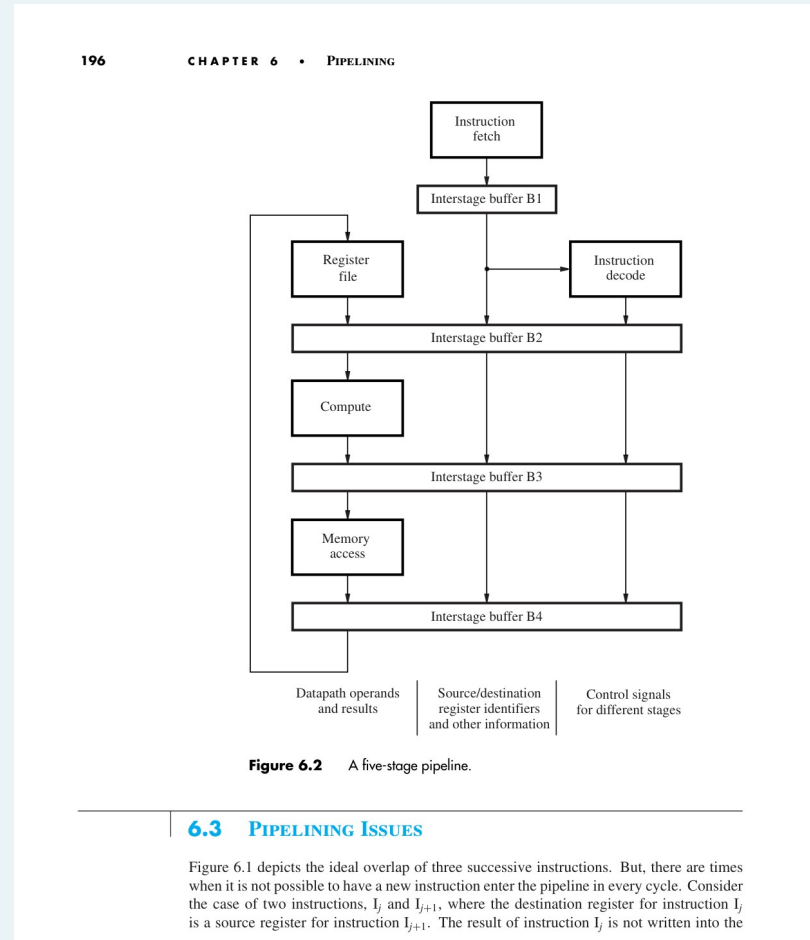


Figure 6.2 - The five steps from Unit 2's single-bus datapath are turned into five hardware stages, each working on a different instruction.

Pipeline Performance

- In an ideal k -stage pipeline processing n instructions, total time = $(k + n - 1)$ cycles, versus $k \times n$ cycles with no overlap.
- Speedup approaches k (the number of stages) as n becomes large - more stages can mean more potential speedup.
- In practice, hazards force the pipeline to stall, which reduces the achieved speedup below the ideal value.
- Performance is also limited by the slowest stage - clock cycle time must accommodate the stage with the longest delay.



PART B

Hazards: Data and Instruction

Data Hazards

- A data hazard occurs when an instruction needs a value that a previous, still-in-flight instruction has not yet produced.
- Example: Add R2, R1 followed by Sub R4, R2 - the Sub needs R2 before the Add has written it back.
- Solution 1 - Operand forwarding: route the ALU result directly to the next instruction's input, bypassing the register file write/read delay.
- Solution 2 - Stalling: hold back (bubble) the dependent instruction until the value is available.
- Compilers can also help by reordering independent instructions between the dependent pair (instruction scheduling).

Data Forwarding Hardware

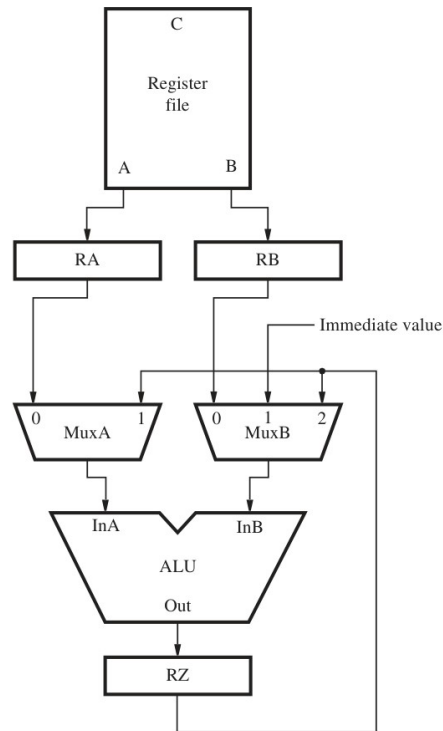


Figure 6.5 Modification of the datapath of Figure 5.8 to support data forwarding from register RZ to the ALU inputs.

Figure 6.5 - Extra forwarding paths feed the ALU result back into MuxA/MuxB so a dependent instruction need not wait for a register write.

Role of Cache Memory in the Pipeline

- A pipeline issues a new instruction (and often a new memory operand access) every cycle - main memory alone is far too slow to keep up.
- Cache memory, introduced briefly in Unit 1 and detailed in Unit 4, supplies most instruction and data accesses at near-processor speed.
- A cache miss stalls the pipeline (a memory-induced hazard) until the needed word/block is fetched from main memory.
- Because cache hit rates are high, the pipeline can normally proceed at close to one instruction per cycle - cache design is therefore central to pipeline performance.

Instruction (Control) Hazards

- A control hazard arises from branch instructions: the pipeline does not know the next instruction's address until the branch is resolved.
- Naive solution: stall the pipeline until the branch outcome and target are known - wastes cycles.
- Branch prediction: guess the outcome (taken/not-taken) and speculatively fetch along the predicted path.
- Delayed branching: define the branch to take effect after a fixed number of following instructions, which are always executed regardless of outcome.
- A mispredicted branch requires flushing the incorrectly fetched instructions from the pipeline - a misprediction penalty.

Effect of a Branch Instruction on the Pipeline

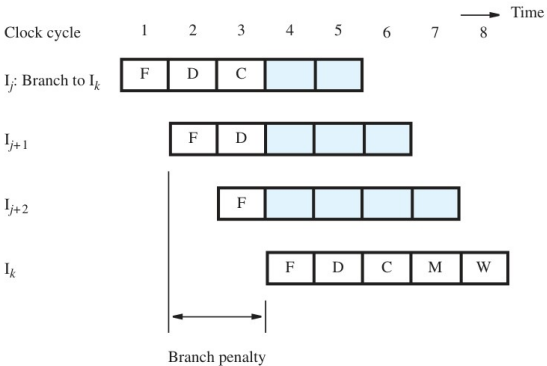


Figure 6.9 Branch penalty when the target address is determined in the Compute stage of the pipeline.

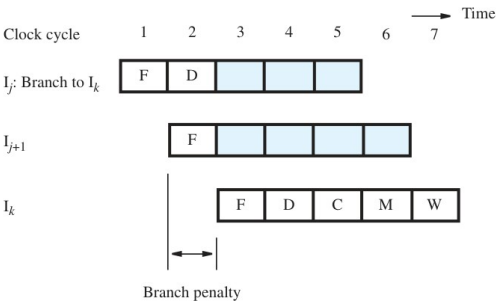
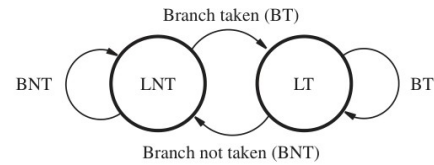


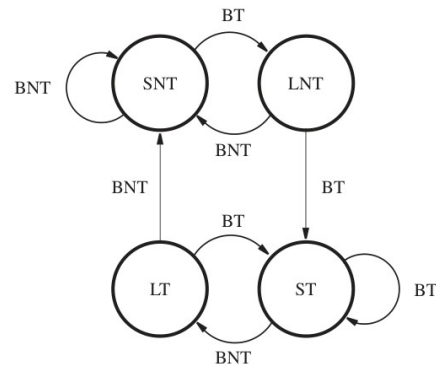
Figure 6.10 Branch penalty when the target address is determined in the Decode stage of the pipeline.

Figure 6.9 - Instructions fetched after a branch may need to be discarded once the branch outcome is known.

Branch Prediction as a State Machine



(a) A 2-state algorithm



(b) A 4-state algorithm

Figure 6.12 State-machine representation of branch prediction algorithms.

Figure 6.12 - A simple 2-bit predictor that strengthens or weakens its taken/not-taken guess based on recent branch history.



PART C

Instruction Sets, Datapath & Superscalar Design

Influence of Pipelining on Instruction Set Design

- Uniform instruction length and a small number of formats make Fetch and Decode stages simpler and faster.
- Limiting memory operands to Load/Store instructions only (Load-Store architecture) keeps the Execute stage's timing predictable.
- Orthogonal addressing modes (usable consistently for every instruction) simplify the operand-fetch stage's control logic.
- These principles are central to RISC instruction set design, which evolved hand-in-hand with pipelining.

Datapath and Control Considerations

- Each pipeline stage needs its own dedicated hardware (so stages do not block each other) and a buffer/register to hold results for the next stage.
- Control signals must now be generated per-stage and carried along with the instruction as it moves down the pipeline (pipelined control).
- Structural hazards occur when two stages need the same hardware resource (e.g., a single memory port for both instruction fetch and data access) - solved with separate instruction/data caches or extra ports.
- More pipeline stages reduce work-per-stage (allowing a faster clock) but increase the penalty of every hazard - a key design trade-off.

Superscalar Processors

- A superscalar processor issues more than one instruction per clock cycle by duplicating execution units (e.g., two ALUs, a separate Load/Store unit).
- This requires extra hardware to check dependencies between instructions issued in the same cycle, and to dispatch independent instructions to different units.
- Out-of-order execution and reservation stations let independent later instructions execute even if an earlier instruction is stalled.
- Superscalar design extends pipelining's central idea: exploit instruction-level parallelism beyond simple one-instruction-per-cycle overlap.

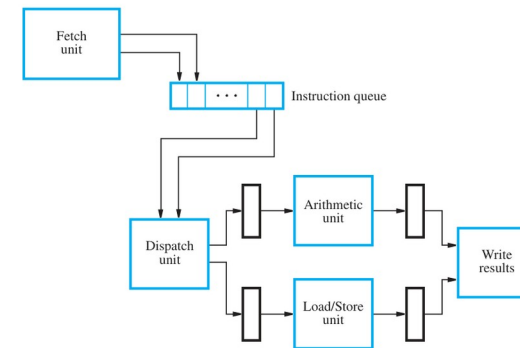


Figure 6.13 A superscalar processor with two execution units.

the simple pipeline. Similarly, an arithmetic instruction and a Load instruction must write their results into the register file when they complete in the same cycle. Thus, the register file must now have two input ports instead of the single input port for the simple pipeline. There is also the potential complication of two instructions completing at the same time with the same destination register for their results. This complication is avoided, if possible, by dispatching the instructions in a manner that prevents its occurrence. Otherwise, one instruction is stalled to ensure that results are written into the destination register in the same order as in the original instruction sequence of the program.

To illustrate superscalar execution in the processor in Figure 6.13, consider the following sequence of instructions:

Add	R2, R3, #100
Load	R5, 16(R6)
Subtract	R7, R8, R9
Store	R10, 24(R11)

Figure 6.14 shows how these instructions would be executed. The fetch unit fetches two instructions every cycle. The instructions are decoded and their source registers are read in the next cycle. Then, they are dispatched to the arithmetic and Load/Store units. Arithmetic operations can be initiated every cycle. A Load or Store instruction can also be initiated every cycle, because the two-stage pipeline overlaps the address calculation for one Load or Store instruction with the memory access for the preceding Load or Store instruction.

Hazard Types - At a Glance

Hazard Type	Cause	Common Remedy
Data hazard	Instruction needs a result not yet produced	Operand forwarding; stalling; instruction scheduling
Instruction/Control hazard	Branch outcome/target not yet known	Branch prediction; delayed branching; stall+flush
Structural hazard	Two stages need the same hardware resource	Duplicate resources (e.g., separate I-cache/D-cache)

Unit Summary

- ✓ Pipelining overlaps the Fetch/Decode/Execute steps of successive instructions across dedicated hardware stages to raise throughput.
- ✓ Ideal speedup approaches the number of pipeline stages; real speedup is reduced by hazards.
- ✓ Cache memory keeps the pipeline supplied with instructions/data fast enough to avoid frequent stalls.
- ✓ Data hazards are resolved by forwarding or stalling; control hazards by prediction or delayed branching.
- ✓ Instruction set design (uniform formats, Load-Store style) and dedicated per-stage datapath/control hardware make efficient pipelining possible; superscalar design issues multiple instructions per cycle.

Practice Questions

1. Define pipelining and explain why it improves throughput rather than the latency of a single instruction.
2. For a 5-stage pipeline executing 100 instructions with no stalls, estimate the total number of cycles required.
3. Differentiate a data hazard from a control hazard, with one example of each.
4. Explain how operand forwarding removes the need to stall for a register-dependency data hazard.
5. Why does a cache miss act like a pipeline hazard? Which stage is typically affected?
6. How does branch prediction reduce the penalty of control hazards, and what happens on a misprediction?